

Preliminary Results: Random Projections vs PCA on S&P 500 Returns

Phase 0 Smoke Test — single seed, single k , top 100 stocks

Vishal Jha, Adel Sahuc
NYU MS Computer Science

Abstract

This report presents the first end-to-end results from our project on random projections for cross-sectional compression of financial return data, motivated by the SOSA 2018 paper *Simple Analyses of the Sparse Johnson–Lindenstrauss Transform* (Cohen, Jayram, Nelson). It is a Phase 0 smoke test whose primary purpose is to validate the implementation pipeline, not to draw scientific conclusions: results are from a single random seed at a single embedding dimension on a single 100-stock universe. With those caveats stated, the four methods we compare (raw, PCA, dense Gaussian JL, sparse JL) behave exactly as theory predicts, which gives us confidence that the architecture is correct and ready to scale to the full grid in Phase 1.

1 Setup

Data. Daily adjusted close prices and volume for the current S&P 500 (today’s constituents), 2014-01-01 to 2024-12-31, downloaded via `yfinance`. After applying a three-layer selection pipeline (candidate pool $\rightarrow$ rank by avg dollar volume $\rightarrow$ data-integrity and anomaly filters), 452 of 491 candidate tickers survive. We use the `top_100` slice (top 100 by average daily dollar volume) for this preliminary run.

Universe. $N = 100$ stocks; the top 5 by liquidity are Tesla, Apple, NVIDIA, Amazon, Microsoft.

Train/test. Chronological 70/30 split. Train: 1936 days (2014-01-03 to 2021-09-10). Test: 831 days (2021-09-13 to 2024-12-31).

Standardization. Z-score per asset using training-set mean and standard deviation only.

Compression dimension. $k = 20$ (so each method maps $\mathbb{R}^{100} \rightarrow \mathbb{R}^{20}$, a $5\times$ compression).

Methods compared.

1. **raw:** identity (no compression). The reference geometry.
2. **PCA / truncated SVD:** data-adaptive. Fits the top 20 principal directions of the standardized training data.
3. **Dense Gaussian JL:** random matrix $A \in \mathbb{R}^{100 \times 20}$ with $A_{ij} \sim \mathcal{N}(0, 1/k)$.
4. **Sparse JL** ($s = 3$): each input column has 3 nonzero entries placed at random columns with random signs $\pm 1/\sqrt{s}$.

Seeds. A single seed (`seed=0`) for both random projection construction and pair sampling. The full Phase 1 experiment will average across 10 seeds per the proposal’s fairness rule.

2 Metric: Pairwise Distance Distortion

2.1 What

For two test days i, j , define the raw-space distance and the compressed-space distance:

$$d_{ij} = \|\tilde{X}_i - \tilde{X}_j\|_2, \quad \hat{d}_{ij} = \|Z_i - Z_j\|_2,$$

where $\tilde{X}$ is the standardized return matrix and Z is its compressed image. The multiplicative distortion is

$$\rho_{ij} = \frac{\hat{d}_{ij}}{d_{ij}}.$$

A perfect distance-preserving embedding has $\rho_{ij} = 1$. We report:

- mean absolute distortion: $\frac{1}{M} \sum_{(i,j)} |\rho_{ij} - 1|$,
- median absolute distortion,
- 95th-percentile absolute distortion,
- mean of ρ_{ij} (signed).

2.2 Why

The Johnson–Lindenstrauss lemma states that for any set of n points in $\mathbb{R}^N$ and any $\epsilon \in (0, 1)$, a random linear map into $\mathbb{R}^k$ with $k = O(\epsilon^{-2} \log n)$ approximately preserves all pairwise Euclidean distances:

$$(1 - \epsilon) d_{ij} \leq \hat{d}_{ij} \leq (1 + \epsilon) d_{ij} \quad \text{for all } i, j \text{ with high probability.}$$

In other words, the JL lemma’s natural success metric is precisely $|\rho_{ij} - 1|$ being small. This is the most direct empirical test of whether the JL machinery actually delivers what its theorem promises in our setting. It is also a natural quality measure for compression in general: any downstream task that depends on pairwise relationships (clustering, nearest-neighbor search, anomaly detection) inherits its quality from how well distances are preserved.

2.3 How

The 831 test days yield $\binom{831}{2} \approx 345\text{k}$ unique day-pairs. We sample 50,000 of them uniformly at random with a fixed seed for reproducibility. For each method, we compute d_{ij} on the raw standardized test matrix and $\hat{d}_{ij}$ on its compressed counterpart, then aggregate the four statistics above. Pair indices are shared across all four methods so the comparison is apples-to-apples.

2.4 What it does *not* measure

A note on scope: distance distortion is one of five metrics specified in the project proposal (the others are nearest-neighbor preservation, clustering stability via Adjusted Rand Index, anomaly recall, and runtime / memory). Distance preservation is the most fundamental, so it goes first, but it does not capture ordering effects (a method could compress all distances by a constant factor and look bad on $|\rho - 1|$ while still preserving every nearest-neighbor relation). The full Phase 1 run will report all five.

3 Results

Table 1: Pairwise distance distortion at $k = 20$ on the `top_100` universe ($N = 100$). 50 000 sampled test-day pairs, single seed.

Method	mean $ \rho - 1 $	median	p95	mean ρ	nonzeros
raw	0.0000	0.0000	0.0000	1.0000	— (identity)
PCA	0.2580	0.2523	0.4443	0.7420	data-adaptive
Dense Gaussian JL	0.1242	0.1059	0.3020	1.0082	2 000
Sparse JL ($s = 3$)	0.1147	0.0969	0.2821	0.9756	300

PCA additionally captures **71% of total variance** in the top 20 components, indicating strong low-rank structure in the standardized return matrix.

4 Interpretation: theory and practice

Dense Gaussian JL behaves like the textbook says. With $\mathbb{E}[\hat{d}_{ij}^2] = d_{ij}^2$ for the Gaussian projection, we expect $\rho \approx 1$ on average. We observe mean $\rho = 1.0082$, well within sampling variation for 50 000 pairs. Mean $|\rho - 1| = 0.124$ is in the expected range for $k = 20$ and $N = 100$.

Sparse JL approximates dense JL with $6.7\times$ fewer parameters. The dense projection has $N \cdot k = 2,000$ nonzero entries; the sparse projection at $s = 3$ has only $N \cdot s = 300$. Despite this, both produce comparable distance distortion (mean $|\rho - 1|$ of 0.115 vs 0.124 in this single-seed run; in expectation across seeds they should be statistically indistinguishable at this s and k). This is exactly the empirical pattern the SOSA paper’s analysis predicts: sparse JL transforms retain the distance-preservation guarantees of dense JL while using a fraction of the storage and compute.

PCA preserves variance, not pairwise distances. PCA finds the rank-20 subspace that captures the most variance in the training data (V_{20}). Distances measured in the projection XV_{20} are smaller than in the original space because they ignore the variance lying in the orthogonal complement (29% of total variance, from the explained-variance figure above). Hence the systematic mean $\rho = 0.742$ — a 26% under-estimate, on average. PCA is not failing here; it is correctly executing its objective, which is different from what the JL methods are optimizing.

The tension visible in this table is the entire research question. Sparse JL gives smaller distance distortion than PCA at this (N, k) . PCA gives a much higher fraction of explained variance. Which one a downstream task prefers depends on whether the task cares about pairwise distance fidelity or about variance capture. For clustering on a low-rank factor space, PCA might still win on Adjusted Rand Index even though it loses on $|\rho - 1|$. For nearest-neighbor recall on rare/unusual market days, JL’s distance preservation may matter more. The Phase 1 multi-metric grid will resolve this question across k , sparsity s , COVID stress slices, and synthetic factor-rank settings.

Practical implications already visible. Sparse JL’s $6.7\times$ sparsity (and the corresponding speedup in $\tilde{X}S$ matrix-vector products) is potentially attractive even when JL is not the most accurate method. If sparse JL is within a few percentage points of dense JL or PCA on the metric the user cares about, its lower compute and memory costs may dominate the trade-off. Whether that condition actually holds in financial data is one of the things the full experiment will test.

5 Limitations of this preliminary run

This is a Phase 0 smoke test, not a result. It is limited in five important ways:

1. **Single seed.** JL methods are randomized. The proposal’s fairness rule (§6.7) requires averaging across 10 seeds. Sparse JL’s marginal advantage over dense JL in the single-seed table above is plausibly noise, not signal.
2. **Single embedding dimension.** The story will look different at $k = 5$ (where PCA likely wins clearly because of low-rank structure) and at $k = 100$ (where it degenerates to identity for raw and to large embedding for JL). The full Phase 1 sweep covers $k \in \{5, 10, 20, 30, 50, 100\}$.
3. **Single universe size.** We only ran $N = 100$. The proposal also specifies `top_200` ($N = 200$) and `all_survivors` ($N = 452$). JL’s relative advantage depends on the input dimension N .
4. **One metric.** Only distance distortion. Nearest-neighbor preservation, clustering stability, anomaly recall, and runtime/memory remain to be implemented.
5. **One evaluation slice.** The full sample including COVID. The proposal also specifies excluding-COVID and COVID-only slices, plus volatility-regime conditioning.

The architecture is verified: each method runs cleanly, schemas line up, the metric returns sensible numbers consistent with theory. With the implementation now trustworthy, the rest of the project is sweep-and-aggregate work, not architectural design.

6 Next steps

Phase 1 turns this single row of the results table into the full grid:

- Sweep $k \in \{5, 10, 20, 30, 50, 100\}$ for all four methods.
- Sweep sparse JL $s \in \{1, 3, 5\}$.
- Average JL methods across 10 random seeds and report mean $\pm$ standard deviation.

- Implement and run the remaining four metrics.
- Add evaluation slices: full-including-COVID, full-excluding-COVID, COVID-only.
- Record runtime and projection-matrix nonzero counts for the runtime/sparsity plots.

This is sweep-and-tabulate, no further architectural decisions required.

Reference. Michael B. Cohen, Thathachar S. Jayram, and Jelani Nelson. *Simple Analyses of the Sparse Johnson–Lindenstrauss Transform*. In *Proceedings of the 1st Symposium on Simplicity in Algorithms (SOSA)*, 2018.

Reproducibility. Source code and exact commands to reproduce: see `dev-notes/recreate-results.md` in the repository.